

Clase 6b - Introducción al uso y aplicación de paquetes estadísticos para el tratamiento de fuentes de información en investigación social cuantitativa

José Rodríguez de la Fuente - Albano Vergara Parra

Tabla de contenidos

Construcción de tablas	1
La función <code>table()</code>	2
Algunas librerías para la construcción de tablas	6
Tablas con <code>dplyr</code> y <code>tidyr</code>	9
El formato <i>tidy</i>	12
Medidas de asociación	13
Exportando los resultados	15
Exportando a <code>xlsx</code>	15
Exportando a <code>docx</code>	18

Construcción de tablas

En esta apartado revisaremos distintas aproximaciones a la construcción de tablas de frecuencias y de tablas de contingencia. Quizás R no se destaque por la simpleza del análisis exploratorio a través de tablas, pero existen infinidad de paquetes que nos permitirán llegar a resultados similares. Empezaremos explorando la funciones disponibles en **R base**, luego revisaremos algunos paquetes que harán la mayor parte del trabajo por nosotros y, por último, veremos cómo podemos hacerlo con `dplyr` y `tidyr` de **tidyverse**.

Para este apartado utilizaremos la base de graduados universitarios de 2016-2018.

```
library(tidyverse)

graduadx_universit <-
  read_csv("https://datos.produccion.gob.ar/dataset/46df1ebe-d0bb-49a1-96dd-fe9751930682/r")
```

La función table()

La función `table()` es una de las más utilizadas para la construcción de tablas de frecuencias, tanto para el análisis univariable como bivariable. Si bien la base de datos de graduados tiene información sobre los egresados universitarios de 2016 a 2018, los datos de ingresos corresponden a la información del *SIPA* (Sistema Integrado Previsional Argentino) para los años 2019, 2020, 2021. Por lo cual, cada egresado está representado tres veces en la base para cada uno de esos años en los que se cuenta con su información salarial. Vamos a quedarnos solo con los datos de 2021, creando un nuevo objeto llamado `graduadx_2021`.

```
graduadx_2021 <- graduadx_universit %>%
  filter(año == 2021)
```

Vamos a identificar las variables de *rama de conocimiento en el que se graduó la persona* (`rama_id`), *disciplina* (`disciplina_id`), *región* (`region_id`) y el *género* (`genero_id`) y las convertiremos en factor. Para ello, utilizaremos el [diccionario](#) de la base de datos para identificar y copiar las categorías de las variables.

```
graduadx_2021 <- graduadx_2021 %>%
  mutate(rama_f = factor(rama_id,
                        labels = c("Ciencias Sociales",
                                   "Ciencias Aplicadas",
                                   "Ciencias de la Salud",
                                   "Ciencias Humanas",
                                   "Ciencias Básicas",
                                   "Sin Rama")),
         disciplina_f = factor(disciplina_id,
                              labels = c("Economía y Administración",
                                           "Ingeniería",
                                           "Sociología, Antropología y
                                           Servicio Social",
                                           "Ciencias Agropecuarias",
                                           "Medicina",
                                           "Ciencias Políticas, Relaciones
                                           Internacionales y Diplomacia",
```

```

"Derecho",
"Artes",
"Educación",
"Informática",
"Psicología",
"Historia",
"Arquitectura y Diseño",
"Letras e Idiomas",
"Meteorología",
"Biología",
"Bioquímica y Farmacia",
"Paramédicas y Auxiliares de la
↳ Medicina",
"Ciencias de la Información y de la
↳ Comunicación",
"Filosofía",
"Industrias",
"Demografía y Geografía",
"Ciencias del Suelo",
"Relaciones Institucionales y
↳ Humanas",
"Salud Pública",
"Veterinaria",
"Otras Ciencias Sociales",
"Otras Ciencias Aplicadas",
"Odontología",
"Física",
"Sanidad",
"Química",
"Teología",
"Matemática",
"Estadística",
"Astronomía",
"Sin Disciplina",
"Arqueología")),

region_f = factor(region_id,
                    labels = c("CABA",
                               "BUENOS AIRES",
                               "RESTO PAMPEANA",
                               "NOA",
                               "NEA",
                               "CUYO",
                               "PATAGONIA")),

```

```
genero_f = factor(genero_id,
                  labels = c("Mujer", "Varón"))
```

Ahora si, con la función `table()` vamos a construir una tabla de frecuencias de la variable 'rama_f'. Como se verá, la salida es rápida pero con información escueta: no tenemos frecuencias relativas, ni acumuladas.

```
table(graduadx_2021$rama_f)
```

Ciencias Sociales	Ciencias Aplicadas	Ciencias de la Salud
112068	64261	50597
Ciencias Humanas	Ciencias Básicas	Sin Rama
38231	7055	1233

Si queremos calcular las proporciones o frecuencias relativas, podemos llamar a la función `prop.table()` sobre la función ya especificada:

```
prop.table(table(graduadx_2021$rama_f))
```

Ciencias Sociales	Ciencias Aplicadas	Ciencias de la Salud
0.409837444	0.235005211	0.185035382
Ciencias Humanas	Ciencias Básicas	Sin Rama
0.139812394	0.025800435	0.004509133

Por último, si queremos presentar los resultados en porcentajes, lo multiplicamos por 100:

```
prop.table(table(graduadx_2021$rama_f)) * 100
```

Ciencias Sociales	Ciencias Aplicadas	Ciencias de la Salud
40.9837444	23.5005211	18.5035382
Ciencias Humanas	Ciencias Básicas	Sin Rama
13.9812394	2.5800435	0.4509133

Estas funciones nos sirven también para tablas cruzadas. Por ejemplo, calculemos el cruce entre las variable rama de conocimiento y género.

```
table(graduadx_2021$rama_f, graduadx_2021$genero_f)
```

	Mujer	Varón
Ciencias Sociales	67615	44453
Ciencias Aplicadas	26347	37914
Ciencias de la Salud	36831	13766
Ciencias Humanas	29251	8980
Ciencias Básicas	4523	2532
Sin Rama	698	535

Y si quisiéramos calcular los porcentajes por columna, utilizamos la opción `margin = 2`.

```
prop.table(table(graduadx_2021$rama_f, graduadx_2021$genero_f), margin = 2)
#>      * 100
```

	Mujer	Varón
Ciencias Sociales	40.9130790	41.0916990
Ciencias Aplicadas	15.9422745	35.0471436
Ciencias de la Salud	22.2860255	12.7250878
Ciencias Humanas	17.6994524	8.3009798
Ciencias Básicas	2.7368166	2.3405435
Sin Rama	0.4223520	0.4945461

Como verán, la función `table()` es muy útil para la construcción rápida de tablas de frecuencias, pero no nos permite realizar análisis complementarios, ni nos otorgan una salida en formato **tidy**.

💡 Sentido de los porcentajes

Por regla general, cuando en una relación entre dos variables podemos establecer que hay una que funciona como independiente y otra como dependiente, solemos ubicar a la primera en la columna y a la segunda en las filas. Al mismo tiempo, los porcentajes suelen calcularse en el sentido de la variable independiente, quedando el 100% en la fila final.

Esto no invalida que, en algunos casos, podamos calcular los porcentajes por fila. Por ejemplo, si queremos conocer la distribución de género por rama de conocimiento.

Algunas librerías para la construcción de tablas

A veces necesitamos hacer procesamiento rápido, sin necesidad de escribir mucho código. Si nuestra opción es quedarnos usando R y no ~~huir~~ migrar a otros programas, podemos utilizar algunas librerías que nos facilitarán la construcción de tablas. Les dejamos una lista con la variedad de librerías que pueden utilizar:

- `janitor`
- `expss`
- `summarytools`
- `gmodels`
- `sjmisc`

Vamos a realizar una primera prueba utilizando la librería `sjmisc`. En primer lugar, la función `frq()` nos devuelve una tabla de frecuencias algo más completa que la función `table()`. Por su parte, tiene la opción `sort.frq` que nos permite ordenar las categorías en forma ascendente o descendente. Vamos a probar con la variable `disciplina_f`.

```
install.packages("sjmisc")
```

```
library(sjmisc)

graduadx_2021 %>%
  frq(disciplina_f, sort.frq = "desc")
```

Tabla 1: disciplina_f <category>

val	label	frq	raw.prc	valid.prc	cum.prc
Economía y Administración		48830	17.86	17.86	17.86
Derecho		35339	12.92	12.92	30.78
Paramédicas y Auxiliares de la Medicina		25039	9.16	9.16	39.94
Medicina		17265	6.31	6.31	46.25
Arquitectura y Diseño		17034	6.23	6.23	52.48
Educación		15383	5.63	5.63	58.11
Ingeniería		14783	5.41	5.41	63.51
Industrias		13187	4.82	4.82	68.34
Psicología		11324	4.14	4.14	72.48
Ciencias de la Información y de la Comunicación		7967	2.91	2.91	75.39
Informática		7725	2.83	2.83	78.22
Sociología, Antropología y Servicio Social		7084	2.59	2.59	80.81
Ciencias Agropecuarias		5305	1.94	1.94	82.75
Artes		4751	1.74	1.74	84.48

Bioquímica y Farmacia	4574	1.67	1.67	86.16
Relaciones Institucionales y Humanas	4508	1.65	1.65	87.80
Letras e Idiomas	4141	1.51	1.51	89.32
Odontología	4132	1.51	1.51	90.83
Biología	3986	1.46	1.46	92.29
Demografía y Geografía	3185	1.16	1.16	93.45
Ciencias Políticas, Relaciones Internacionales y Diplomacia	2967	1.09	1.09	94.54
Veterinaria	2945	1.08	1.08	95.61
Otras Ciencias Sociales	2188	0.80	0.80	96.42
Química	1665	0.61	0.61	97.02
Historia	1608	0.59	0.59	97.61
Sin Disciplina	1233	0.45	0.45	98.06
Sanidad	874	0.32	0.32	98.38
Matemática	854	0.31	0.31	98.69
Filosofía	810	0.30	0.30	98.99
Ciencias del Suelo	796	0.29	0.29	99.28
Otras Ciencias Aplicadas	621	0.23	0.23	99.51
Física	550	0.20	0.20	99.71
Salud Pública	342	0.13	0.13	99.84
Teología	176	0.06	0.06	99.90
Estadística	119	0.04	0.04	99.94
Meteorología	66	0.02	0.02	99.97
Astronomía	51	0.02	0.02	99.99
Arqueología	38	0.01	0.01	100.00
NA	NA	0	0.00	NA
total N=273445 · valid N=273445 · $\bar{x}$ =10.42 · =8.21				

El mismo paquete nos permite realizar tablas cruzadas con la función `flat_table`.

```
graduadx_2021 %>%
  flat_table(rama_f, genero_f, margin = "row")
```

	genero_f	Mujer	Varón
rama_f			
Ciencias Sociales		60.33	39.67
Ciencias Aplicadas		41.00	59.00
Ciencias de la Salud		72.79	27.21
Ciencias Humanas		76.51	23.49
Ciencias Básicas		64.11	35.89
Sin Rama		56.61	43.39

`summarytools` es otra librería que nos permite realizar tablas de frecuencias. En este caso, la función `freq()` nos devuelve una tabla de frecuencias similar a la función anterior.

```
library(summarytools)

graduadx_2021 %>%
  freq(rama_f)
```

Frequencies
graduadx_2021\$rama_f
Type: Factor

	Freq	% Valid	% Valid Cum.	% Total	% Total Cum.
Ciencias Sociales	112068	40.98	40.98	40.98	40.98
Ciencias Aplicadas	64261	23.50	64.48	23.50	64.48
Ciencias de la Salud	50597	18.50	82.99	18.50	82.99
Ciencias Humanas	38231	13.98	96.97	13.98	96.97
Ciencias Básicas	7055	2.58	99.55	2.58	99.55
Sin Rama	1233	0.45	100.00	0.45	100.00
<NA>	0			0.00	100.00
Total	273445	100.00	100.00	100.00	100.00

Del mismo modo, la función `ctable()` realiza el cruce entre ambas variables, pudiendo elegir calcular las proporciones por fila, columna o total con la opción `prop`. El problema es que esta función no funciona con el *pipe* `%>%`.

```
ctable(ggraduadx_2021$rama_f, graduadx_2021$genero_f, prop = "r")
```

Cross-Tabulation, Row Proportions
rama_f * genero_f
Data Frame: graduadx_2021

	genero_f	Mujer	Varón	Total
rama_f				
Ciencias Sociales		67615 (60.3%)	44453 (39.7%)	112068 (100.0%)
Ciencias Aplicadas		26347 (41.0%)	37914 (59.0%)	64261 (100.0%)
Ciencias de la Salud		36831 (72.8%)	13766 (27.2%)	50597 (100.0%)
Ciencias Humanas		29251 (76.5%)	8980 (23.5%)	38231 (100.0%)
Ciencias Básicas		4523 (64.1%)	2532 (35.9%)	7055 (100.0%)

Sin Rama	698 (56.6%)	535 (43.4%)	1233 (100.0%)
Total	165265 (60.4%)	108180 (39.6%)	273445 (100.0%)

Tablas con dplyr y tidyr

Si bien las librerías anteriores nos permiten realizar tablas de frecuencias de forma rápida y sencilla, no siguen la filosofía del **tidyverse** que venimos utilizando y que nos servirá para trabajar en forma ordenada en cada momento de la investigación. Quizás, a principio, la elaboración de tablas con estas funciones sea algo complicado, pero a futuro nos facilitará la posterior presentación de los resultados en documentos o en gráficos.

La secuencia de pasos es similar a como lo hemos hecho en el apartado de **agregación de datos**. Primero agrupamos la información (`group_by()`) y luego contamos los casos. Esto último podemos hacerlo mediante la función `summarise()`, `count()` o `tally()`.

La función `count()` no necesita de la función `group_by()`, ya que cuenta los casos por grupos automáticamente. Por su parte, `tally()` necesita que previamente agrupemos la información con `group_by()`, pero a futuro nos permitirá hacer conteos ponderados.

```
graduadx_2021 %>%
  group_by(rama_f) %>%
  summarise(frecuencia = n())
```

```
# A tibble: 6 x 2
  rama_f      frecuencia
  <fct>      <int>
1 Ciencias Sociales 112068
2 Ciencias Aplicadas 64261
3 Ciencias de la Salud 50597
4 Ciencias Humanas 38231
5 Ciencias Básicas 7055
6 Sin Rama 1233
```

```
graduadx_2021 %>%
  count(rama_f)
```

```
# A tibble: 6 x 2
  rama_f      n
  <fct>      <int>
1 Ciencias Sociales 112068
```

2	Ciencias Aplicadas	64261
3	Ciencias de la Salud	50597
4	Ciencias Humanas	38231
5	Ciencias Básicas	7055
6	Sin Rama	1233

```
graduadx_2021 %>%
  group_by(rama_f) %>%
  tally()
```

```
# A tibble: 6 x 2
  rama_f      n
  <fct>    <int>
1 Ciencias Sociales 112068
2 Ciencias Aplicadas 64261
3 Ciencias de la Salud 50597
4 Ciencias Humanas 38231
5 Ciencias Básicas 7055
6 Sin Rama 1233
```

Para realizar tablas bivariadas, lo primero que adicionaremos es la segunda variable a la función `count()`.

```
graduadx_2021 %>%
  count(rama_f, genero_f)
```

```
# A tibble: 12 x 3
  rama_f      genero_f      n
  <fct>    <fct>    <int>
1 Ciencias Sociales Mujer 67615
2 Ciencias Sociales Varón 44453
3 Ciencias Aplicadas Mujer 26347
4 Ciencias Aplicadas Varón 37914
5 Ciencias de la Salud Mujer 36831
6 Ciencias de la Salud Varón 13766
7 Ciencias Humanas Mujer 29251
8 Ciencias Humanas Varón 8980
9 Ciencias Básicas Mujer 4523
10 Ciencias Básicas Varón 2532
11 Sin Rama Mujer 698
12 Sin Rama Varón 535
```

Como verán, la tabla se nos presenta en un formato extraño, que se denomina *long* o largo, debido a que las dos variables de interés se sitúan en las primeras columnas y las frecuencias en la última. En algunos casos, como por ejemplo para la construcción de gráficos, este formato es apropiado. Pero para la presentación de tablas, necesitamos un formato de tipo *wide* o ancho.

Para presentar la tabla en un formato más amigable, utilizaremos la función `pivot_wider` de `tidyr`. Lo que debemos declarar en dicha función es cual es la variable que pasará a mostrar sus categorías en las columnas y de dónde surgirán los valores que irán en las celdas. En nuestro caso, las columnas se completarán con las categorías de la variable `genero_f` y los valores serán las frecuencias de los conteos de la variable `n` que construimos en el paso anterior.

```
graduadx_2021 %>%  
  count(rama_f, genero_f) %>%  
  pivot_wider(names_from = genero_f, values_from = n)
```

```
# A tibble: 6 x 3  
  rama_f      Mujer Varón  
  <fct>      <int> <int>  
1 Ciencias Sociales  67615 44453  
2 Ciencias Aplicadas 26347 37914  
3 Ciencias de la Salud 36831 13766  
4 Ciencias Humanas   29251  8980  
5 Ciencias Básicas    4523  2532  
6 Sin Rama           698   535
```

Como se vera, la tabla se presenta de forma más ordenada y clara, sin embargo, aún no hemos calculado las proporciones. Para ello, lo mejor es valerse de la librería `janitor` que automáticamente nos calculará distintos tipos de proporciones, nos presentará los datos en formato porcentual y nos dejará agregar los totales por fila o columna. Exploreemos la librería.

```
install.packages("janitor")
```

```
library(janitor)  
  
graduadx_2021 %>%  
  count(rama_f, genero_f) %>%  
  pivot_wider(names_from = genero_f, values_from = n) %>%  
  adorn_totals("row") %>% # Agregamos los totales por fila  
  adorn_percentages("row") %>% # Calculamos las proporciones por fila  
  adorn_pct_formatting(digits = 1) # Presentamos los valores en formato  
  percent
```

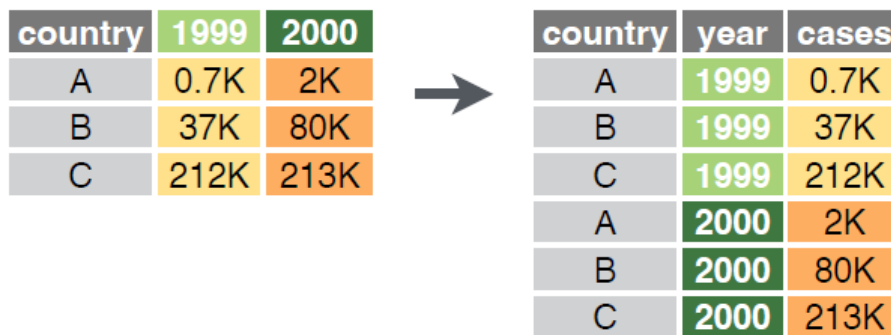
rama_f	Mujer	Varón
Ciencias Sociales	60.3%	39.7%
Ciencias Aplicadas	41.0%	59.0%
Ciencias de la Salud	72.8%	27.2%
Ciencias Humanas	76.5%	23.5%
Ciencias Básicas	64.1%	35.9%
Sin Rama	56.6%	43.4%
Total	60.4%	39.6%

El formato *tidy*

Disponer los datos en forma ordenada implica que: 1) cada variable es una columna, 2) cada observación es una fila y 3) cada valor se encuentra en una celda. Sin embargo muchas veces, los datos distan de presentarse en dicho formato. Por ejemplo, es frecuente que:

- los encabezados de las columnas sean valores,
- muchas variables sean guardadas en una columna,
- las variables se encuentren en las filas y columnas,
- múltiples unidades de análisis se encuentren en una misma tabla,
- una unidad de análisis se encuentre en varias tablas.

Para resolver estos problemas, `tidyr` nos ofrece las funciones `pivot_longer()` y `pivot_wider()`. La primera nos permite pasar de un formato ancho a uno largo, mientras que la segunda nos permite pasar de un formato largo a uno ancho.



country	1999	2000
A	0.7K	2K
B	37K	80K
C	212K	213K

country	year	cases
A	1999	0.7K
B	1999	37K
C	1999	212K
A	2000	2K
B	2000	80K
C	2000	213K

Figura 1: Formato ancho

`pivot_longer()` nos permite pasar de un formato ancho a uno largo. Para ello, debemos especificar las columnas que queremos mantener y las que queremos transformar en filas. Con la función `names_to` especificamos el nombre de la nueva columna que contendrá los nombres de las columnas que estamos transformando en filas, y con `values_to` especificamos el nombre

de la nueva columna que contendrá los valores de las columnas que estamos transformando en filas.

country	year	type	count		country	year	cases	pop
A	1999	cases	0.7K	→	A	1999	0.7K	19M
A	1999	pop	19M		A	2000	2K	20M
A	2000	cases	2K		B	1999	37K	172M
A	2000	pop	20M		B	2000	80K	174M
B	1999	cases	37K		C	1999	212K	1T
B	1999	pop	172M		C	2000	213K	1T
B	2000	cases	80K					
B	2000	pop	174M					
C	1999	cases	212K					
C	1999	pop	1T					
C	2000	cases	213K					
C	2000	pop	1T					

Figura 2: Formato largo

`pivot_wider()` nos permite pasar de un formato largo a uno ancho. Para ello, debemos especificar las columnas que queremos mantener y las que queremos transformar en columnas. Con la función `names_from` especificamos el nombre de la nueva columna que contendrá los nombres de las columnas que estamos transformando en columnas, y con `values_from` especificamos el nombre de la nueva columna que contendrá los valores de las columnas que estamos transformando en columnas.

Medidas de asociación

Las medidas de asociación son estadísticas que nos permiten evaluar la relación entre dos variables. En el caso de las tablas de contingencia, las medidas de asociación nos permiten evaluar si existe una relación entre las variables y, en caso de existir, cuál es la fuerza y dirección de esa relación (en el caso de variables ordinales).

Existen muchas medidas de asociación, pero aquí señalaremos algunas de las más comunes y frecuentemente utilizadas:

- **Chi-cuadrado de Pearson:** es una medida de asociación que nos permite evaluar si existe una relación entre dos variables categóricas. La hipótesis nula es que no existe relación entre las variables. Si el valor de chi-cuadrado es significativo, rechazamos la

hipótesis nula y concluimos que existe una relación entre las variables. Su mínimo es 0 y su máximo es infinito. No mide la fuerza de la asociación.

- **V de Cramer:** está basada en la prueba de Chi cuadrado y está pensada para variables nominales. Permite determinar la fuerza de la asociación. Su valor oscila entre 0 y 1. A mayor valor, mayor es la fuerza de la asociación.
- **Tau de Kendall:** es una medida de asociación no paramétrica que mide la asociación entre dos variables ordinales. Su valor oscila entre -1 y 1. Valores positivos indican una asociación directa, valores negativos una asociación inversa y 0 la ausencia de asociación.
- **Residuos:** los residuos son una medida de asociación que nos permite identificar en qué celdas de la tabla de contingencia se concentra la asociación. Los residuos positivos indican que la frecuencia observada es mayor que la esperada, mientras que los residuos negativos indican que la frecuencia observada es menor que la esperada.

Vamos a calcular el chi-cuadrado de Pearson y la V de Cramer para evaluar la asociación existente entre la rama de conocimiento (`rama_f`) y el género (`genero_f`). Para ello, utilizaremos la función `chisq.test()` de R `base` y la función `CramerV` de la librería `DescTools`.

```
install.packages("DescTools")
```

```
library(DescTools)

# Calculamos el chi-cuadrado de Pearson
chisq.test(graduadx_2021$rama_f, graduadx_2021$genero_f)
```

Pearson's Chi-squared test

```
data: graduadx_2021$rama_f and graduadx_2021$genero_f
X-squared = 17563, df = 5, p-value < 2.2e-16
```

```
# Calculamos otros coeficientes
CramerV(graduadx_2021$rama_f, graduadx_2021$genero_f)
```

```
[1] 0.2534361
```

El resultado del chi-cuadrado de Pearson nos indica que existe una relación significativa entre las variables, ya que el valor de p es menor a 0.05. Por otro lado, la V de Cramer nos indica que la fuerza de la asociación es alta, ya que el valor es 0.253, es decir, el género influye la rama de conocimiento en la que egresan las personas desde sistema universitario.

Exportando los resultados

Hasta aquí hemos trabajado con tablas de frecuencias y con tablas de contingencia. Si lo que necesitamos es explorar los datos para conocer la información que disponemos, probar hipótesis o realizar análisis descriptivos, con las salidas gráficas que nos ofrece **RStudio** nos bastaría. Sin embargo, a veces, necesitamos presentar los resultados en informes, documentos o presentaciones. Para ello, necesitamos exportar los resultados a formatos comúnmente como `xlsx`, `docx`, `pdf`, etc.

En esta sección vamos a mostrar algunas alternativas para exportar los resultados a una hoja de cálculo (`xlsx`) y a un documento de texto (`docx`). No obstante, existen muchas otras opciones de exportación que pueden explorarse y que se complementan muy bien con el análisis estadístico en **RStudio**, tales como `html` o `pdf`.

💡 Recomendación

A los interesados les recomendamos revisar los sistemas de publicación científica tales como [Quarto](https://quarto.org/). Ambos sistemas permiten la integración de código R con texto, gráficos y tablas, y la exportación a distintos formatos de salida, permitiendo la creación de **documentos reproducibles**.

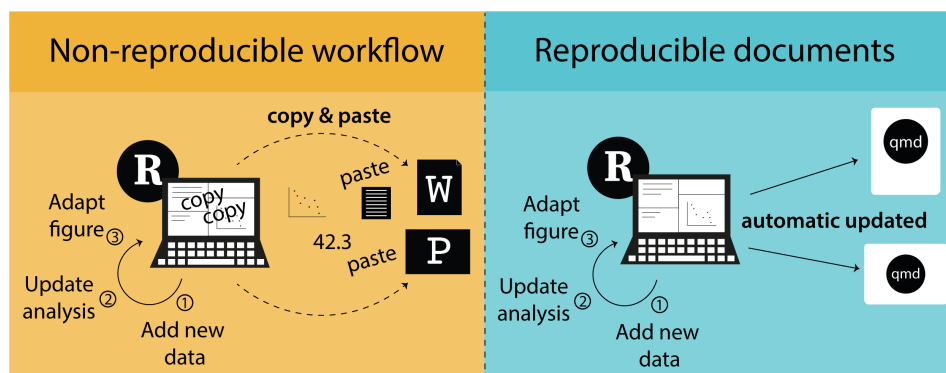


Figura 3: Flujos de trabajo reproducibles y no reproducibles. Fuente: <https://biostats-r.github.io/biostats/quarto/>

Exportando a `xlsx`

Para exportar los resultados a una hoja de cálculo, utilizaremos la librería `openxlsx`. Esta librería nos permite exportar los resultados a un archivo de `Excel`, añadir hojas de cálculo, modificar el formato de las celdas, entre otras cosas. Vamos a revisar las funciones más sencillas del paquete. Para ello, crearemos un objeto con la tabla cruzada que creamos en la sección anterior.

```
install.packages("openxlsx")
```

```
library(openxlsx)

# Creamos un objeto con la tabla de frecuencias
tabla_rama <- graduadxs_2021 %>%
  count(rama_f, region_f) %>%
  pivot_wider(names_from = region_f, values_from = n) %>%
  adorn_totals("row") %>% # Agregamos los totales por fila
  adorn_percentages("col") %>% # Calculamos las proporciones por fila
  adorn_pct_formatting(digits = 1) # Presentamos los valores en formato
  ~ percent

# Exportamos la tabla a un archivo de Excel
write.xlsx(tabla_rama, "resultados/tabla_rama.xlsx")
```

El resultado de la función `write.xlsx` fue la creación de un archivo en la carpeta *resultados* con el nombre *tabla_educ.xlsx*. Si abrimos el archivo, veremos que la tabla se ha exportado correctamente.

	A	B	C	D	E	F	G	H	I	J
1	rama_f	CABA	BUENOS AIRES	RESTO PAMPA	NOA	NEA	CUYO	PATAGONIA		
2	Ciencias Sociales	47.1%	42.9%	35.8%	36.9%	45.2%	36.8%	34.0%		
3	Ciencias Aplicadas	22.2%	23.4%	25.7%	21.8%	17.1%	25.8%	26.5%		
4	Ciencias de la Salud	17.9%	15.7%	20.4%	20.9%	19.2%	17.7%	21.7%		
5	Ciencias Humanas	10.8%	15.5%	14.3%	16.4%	15.2%	16.0%	13.2%		
6	Ciencias Básicas	1.9%	2.5%	2.3%	3.7%	3.3%	3.6%	4.3%		
7	Sin Rama	0.1%	0.0%	1.5%	0.3%	0.0%	0.2%	0.2%		
8	Total	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%	100.0%		
9										
10										
11										
12										
13										
14										
15										
16										
17										

Figura 4: Ejemplo uso `write.xlsx`

Ahora bien, si dentro de nuestro flujo de trabajo construimos varias tablas y queremos exportarlas a un único archivo, podemos hacerlo creando un *workbook* con la función `createWorkbook()` y luego añadiendo las tablas con la función `addWorksheet()`. Veamos paso a paso cómo hacerlo. En la primera hoja pondremos la tabla que creamos anteriormente y en la segunda hoja incorporaremos la tabla que cruza la disciplina por género.

```
# Creamos un objeto con la tabla de disciplina por género
tabla_disciplina <- graduadxs_2021 %>%
  count(disciplina_f, genero_f) %>%
  pivot_wider(names_from = genero_f, values_from = n) %>%
  adorn_totals("row") %>% # Agregamos los totales por fila
  adorn_percentages("col") %>% # Calculamos las proporciones por fila
  adorn_pct_formatting(digits = 1) # Presentamos los valores en formato
  ↪ percent

# Creamos un workbook
wb <- createWorkbook()

# Añadimos la primera hoja
addWorksheet(wb, sheetName = "Disciplina x género")

# Escribimos la tabla en la primera hoja
writeData(wb, sheet = "Disciplina x género", x = "Distribución de la
  ↪ población egresada entre 2016 y 2018 según disciplina académica por
  ↪ género", startRow = 1)
writeData(wb, sheet = "Disciplina x género", x = tabla_disciplina,
  borders = "rows" , borderStyle = "dashed", startRow = 3)

# Añadimos la segunda hoja
addWorksheet(wb, sheetName = "Rama x región")

# Escribimos la tabla en la segunda hoja
writeData(wb, sheet = "Rama x región", x = "Distribución de la población
  ↪ egresada entre 2016 y 2018 según rama de conocimiento por región",
  ↪ startRow = 1)
writeData(wb, sheet = "Rama x región", x = tabla_rama,
  borders = "rows" , borderStyle = "medium", startRow = 3)

# Guardamos el archivo
saveWorkbook(wb, "resultados/tablas_egresados.xlsx", overwrite = TRUE)
```

Como se ve, el archivo ahora tiene dos hojas, y en cada una de ellas encontramos las tablas que exportamos desde RStudio.

	A	B	C	D	E	F	G	H	I
1	Distribución de la población egresada entre 2016 y 2018 según disciplina académica por género								
2									
3	disciplina_f	Mujer	Varón						
4	Economía y Finanzas	16.4%	20.1%						
5	Ingeniería	2.2%	10.3%						
6	Sociología, Antropología	3.4%	1.3%						
7	Ciencias Agrícolas	1.0%	3.4%						
8	Medicina	6.6%	5.8%						
9	Ciencias Políticas	1.0%	1.2%						
10	Derecho	12.8%	13.1%						
11	Artes	1.9%	1.5%						
12	Educación	7.2%	3.2%						
13	Informática	0.8%	5.8%						
14	Psicología	5.7%	1.8%						
15	Historia	0.5%	0.7%						
16	Arquitectura	6.5%	5.9%						
17	Letras e Idiomas	2.1%	0.7%						
18	Meteorología	0.0%	0.0%						

Figura 5: Ejemplo de construcción de hojas en archivo.xlsx

Exportando a docx

Otra de las situaciones en la que nos podemos encontrar es ante la necesidad de exportar las tablas realizadas directamente a un archivo .docx en donde luego escribiremos texto. Para ello, utilizaremos la librería `flextable`, que nos permite no solo exportar las tablas, sino editarlas para que queden en un formato amigable para reportes, documentos, *papers*, etc.

Nuevamente, esta librería funciona en formato *tidy* y permite el uso de *pipes*. Lo primero que vamos a hacer es tomar el objeto `tabla_disciplina` que creamos anteriormente y exportarlo a un archivo .docx. Para esto, utilizaremos primero la función `flextable()` para convertir la tabla en un objeto de tipo *flextable*.

```
install.packages("flextable")
```

```
library(flextable)
```

```
flextable(tabla_disciplina)
```

disciplina_f	Mujer	Varón
Economía y Administración	16.4%	20.1%
Ingeniería	2.2%	10.3%
Sociología, Antropología y Servicio Social	3.4%	1.3%
Ciencias Agropecuarias	1.0%	3.4%
Medicina	6.6%	5.8%
Ciencias Políticas, Relaciones Internacionales y Diplomacia	1.0%	1.2%
Derecho	12.8%	13.1%
Artes	1.9%	1.5%
Educación	7.2%	3.2%
Informática	0.8%	5.8%
Psicología	5.7%	1.8%
Historia	0.5%	0.7%

disciplina_f	Mujer	Varón
Arquitectura y Diseño	6.5%	5.9%
Letras e Idiomas	2.1%	0.7%
Meteorología	0.0%	0.0%
Biología	1.7%	1.1%
Bioquímica y Farmacia	2.1%	0.9%
Paramédicas y Auxiliares de la Medicina	12.4%	4.2%
Ciencias de la Información y de la Comunicación	3.2%	2.5%
Filosofía	0.2%	0.4%
Industrias	2.9%	7.8%
Demografía y Geografía	1.4%	0.8%
Ciencias del Suelo	0.2%	0.4%
Relaciones Institucionales y Humanas	2.1%	1.0%
Salud Pública	0.2%	0.1%
Veterinaria	1.0%	1.3%

disciplina_f	Mujer	Varón
Otras		
Ciencias Sociales	0.6%	1.1%
Otras		
Ciencias Aplicadas	0.1%	0.4%
Odontología	1.8%	1.1%
Física	0.1%	0.4%
Sanidad	0.3%	0.3%
Química	0.6%	0.6%
Teología	0.0%	0.1%
Matemática	0.3%	0.3%
Estadística	0.1%	0.0%
Astronomía	0.0%	0.0%
Sin Disciplina	0.4%	0.5%
Arqueología	0.0%	0.0%
Total	100.0%	100.0%

La librería incluye muchísimas funciones para editar y automatizar el formato de salida de las tablas. Por ejemplo, podemos cambiar el formato de las celdas, agregar títulos, subtítulos, notas al pie, entre otras cosas.

Recomendación

Recomendamos revisar la página de la librería [flextable](#) para conocer las potencialidades que provee.

Vamos a probar ahora agregar en la primera columna la etiqueta de la variable `disciplina_f` y en en una fila superior la etiqueta de la variable `genero_f`. Exploraremos algunas opciones de la función como `set_header_labels()`, `add_header_row()`, `add_footer_lines()`, `set_caption()`, `theme_vanilla()` y `autofit()`. El resultado lo guardaremos en un objeto llamado `tabla_doc`.

```

tabla_doc <- flextable(tabla_disciplina) %>%
  set_header_labels(p09 = "Disciplina") %>% # coloco la etiqueta de la
  ↪ variable disciplina en la primera columna
  add_header_row(values = c("", "Género"), colwidths = c(1,2)) %>% # coloco
  ↪ la etiqueta de la variable género en una fila superior
  add_footer_lines(values = c("Fuente: elaboración propia en base a datos del
  ↪ Centro de Estudios para la Producción (CEP XXI),") %>% # agrego una
  ↪ nota al pie
  set_caption("Distribución de la población egresada entre 2016 y 2018 según
  ↪ disciplina académica por género") %>% # agrego un título a la tabla
  theme_vanilla() %>% # cambio el tema de la tabla
  autofit() # ajusto el ancho de las columnas automáticamente

tabla_doc

```

	Género	
disciplina_f	Mujer	Varón
Economía y Administración	16.4%	20.1%
Ingeniería	2.2%	10.3%
Sociología, Antropología y Servicio Social	3.4%	1.3%
Ciencias Agropecuarias	1.0%	3.4%
Medicina	6.6%	5.8%
Ciencias Políticas, Relaciones Internacionales y Diplomacia	1.0%	1.2%
Derecho	12.8%	13.1%
Artes	1.9%	1.5%
Educación	7.2%	3.2%
Informática	0.8%	5.8%
Psicología	5.7%	1.8%
Historia	0.5%	0.7%
Arquitectura y Diseño	6.5%	5.9%
Letras e Idiomas	2.1%	0.7%
Meteorología	0.0%	0.0%
Biología	1.7%	1.1%

disciplina_f	Género	
	Mujer	Varón
Bioquímica y Farmacia	2.1%	0.9%
Paramédicas y Auxiliares de la Medicina	12.4%	4.2%
Ciencias de la Información y de la Comunicación	3.2%	2.5%
Filosofía	0.2%	0.4%
Industrias	2.9%	7.8%
Demografía y Geografía	1.4%	0.8%
Ciencias del Suelo	0.2%	0.4%
Relaciones Institucionales y Humanas	2.1%	1.0%
Salud Pública	0.2%	0.1%
Veterinaria	1.0%	1.3%
Otras Ciencias Sociales	0.6%	1.1%
Otras Ciencias Aplicadas	0.1%	0.4%
Odontología	1.8%	1.1%
Física	0.1%	0.4%
Sanidad	0.3%	0.3%
Química	0.6%	0.6%
Teología	0.0%	0.1%
Matemática	0.3%	0.3%
Estadística	0.1%	0.0%
Astronomía	0.0%	0.0%
Sin Disciplina	0.4%	0.5%
Arqueología	0.0%	0.0%
Total	100.0%	100.0%
Fuente: elaboración propia en base a datos del Centro de Estudios para la Producción (CEP XXI),		

Finalmente, para exportar la tabla a un archivo .docx, utilizaremos la función `save_as_docx()`,

asignándole la carpeta donde queremos guardar el archivo y el nombre del mismo.

```
save_as_docx(tabla_doc, path = "resultados/tabla_disciplina.docx")
```